```
# ========================
# 1. IMPORTS + DEVICE
# ========================
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# ========================
# 2. DATA (IMPROVED)
# ========================
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=128, shuffle=False)

# ========================
# 3. PRUNABLE LAYER
# ========================
class PrunableLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()

        self.weight = nn.Parameter(torch.randn(out_features, in_features) * 0.02)
        self.bias = nn.Parameter(torch.zeros(out_features))
        self.gate_scores = nn.Parameter(torch.randn(out_features, in_features))

        self.temperature = 1.0

    def forward(self, x):
        gates = torch.sigmoid(self.gate_scores / self.temperature)
        pruned_weights = self.weight * gates
        return F.linear(x, pruned_weights, self.bias)

    def get_gates(self):
        return torch.sigmoid(self.gate_scores / self.temperature)

    def set_temperature(self, temp):
        self.temperature = temp

# ========================
# 4. MODEL (UPGRADED)
# ========================
class PrunableNet(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )

        self.fc1 = PrunableLinear(64 * 8 * 8, 512)  # increased
        self.fc2 = PrunableLinear(512, 10)
```

```
    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# ========================
# 5. SPARSITY LOSS (FIXED)
# ========================
def compute_sparsity_loss(model):
    total_loss = 0.0
    count = 0

    for module in model.modules():
        if hasattr(module, "get_gates"):
            gates = module.get_gates()
            total_loss += torch.mean(gates)  # normalized
            count += 1

    return total_loss / count

# ========================
# 6. SPARSITY METRIC
# ========================
def compute_sparsity(model, threshold=1e-2):
    total = 0
    zero = 0

    for module in model.modules():
        if hasattr(module, "get_gates"):
            gates = module.get_gates()
            total += gates.numel()
            zero += (gates < threshold).sum().item()

    return zero / total

# ========================
# 7. HARD PRUNING
# ========================
def hard_prune(model, threshold=0.1):
    for module in model.modules():
        if hasattr(module, "get_gates"):
            gates = module.get_gates()
            mask = (gates > threshold).float()
            module.weight.data *= mask

# ========================
# 8. TRAIN FUNCTION (UPGRADED)
# ========================
def train(model, epochs=50, base_lambda=1e-3):
    model.to(device)
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(epochs):
        model.train()
        running_loss = 0

        # Smooth lambda increase
        lambda_sparse = base_lambda * (epoch / epochs) ** 2

        # Strong temperature annealing
        temp = max(0.05, 1.0 * (0.95 ** epoch))

        for module in model.modules():
            if hasattr(module, "set_temperature"):
                module.set_temperature(temp)

        for images, labels in trainloader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)
            ce_loss = criterion(outputs, labels)
            sparse_loss = compute_sparsity_loss(model)
```

```
            sparse_loss = compute_sparsity_loss(model)

            loss = ce_loss + lambda_sparse * sparse_loss

            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        sparsity = compute_sparsity(model)
        print(f"Epoch {epoch+1}: Loss={running_loss:.4f}, Sparsity={sparsity:.4f}")

    return model

# ========================
# 9. EVALUATION
# ========================
def evaluate(model):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in testloader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    acc = 100 * correct / total
    sparsity = compute_sparsity(model)

    return acc, sparsity

# ========================
# 10. RUN EXPERIMENTS
# ========================
def run_experiments():
    lambdas = [1e-3, 5e-3, 1e-2]
    results = []

    for lam in lambdas:
        print(f"\nTraining with lambda = {lam}")

        model = PrunableNet()
        model = train(model, epochs=50, base_lambda=lam)

        # Apply hard pruning
        hard_prune(model)

        acc, sparsity = evaluate(model)

        results.append((lam, acc, sparsity))

    print("\nFinal Results:")
    print("Lambda\tAccuracy\tSparsity")

    for lam, acc, sparsity in results:
        print(f"{lam}\t{acc:.2f}%\t\t{sparsity:.2f}")

# ========================
# 11. RUN
# ========================
run_experiments()
```

```
Epoch 10: Loss=202.7594, Sparsity=0.0020
```

```
Epoch 10: Loss=292.7504, Sparsity=0.0020
Epoch 11: Loss=280.5234, Sparsity=0.0031
Epoch 12: Loss=271.8742, Sparsity=0.0046
Epoch 13: Loss=261.7025, Sparsity=0.0067
Epoch 14: Loss=256.3087, Sparsity=0.0095
Epoch 15: Loss=249.3563, Sparsity=0.0129
Epoch 16: Loss=244.1334, Sparsity=0.0172
Epoch 17: Loss=237.8589, Sparsity=0.0224
Epoch 18: Loss=231.2465, Sparsity=0.0284
Epoch 19: Loss=227.3189, Sparsity=0.0353
Epoch 20: Loss=223.8906, Sparsity=0.0431
Epoch 21: Loss=219.3399, Sparsity=0.0518
Epoch 22: Loss=214.2091, Sparsity=0.0612
Epoch 23: Loss=211.5401, Sparsity=0.0713
Epoch 24: Loss=206.0000, Sparsity=0.0820
Epoch 25: Loss=199.7037, Sparsity=0.0934
Epoch 26: Loss=198.7001, Sparsity=0.1053
Epoch 27: Loss=194.4343, Sparsity=0.1177
Epoch 28: Loss=192.7104, Sparsity=0.1305
Epoch 29: Loss=193.2811, Sparsity=0.1436
Epoch 30: Loss=187.3812, Sparsity=0.1570
Epoch 31: Loss=184.2272, Sparsity=0.1704
Epoch 32: Loss=184.7491, Sparsity=0.1841
Epoch 33: Loss=179.4668, Sparsity=0.1980
Epoch 34: Loss=178.4297, Sparsity=0.2120
Epoch 35: Loss=175.2997, Sparsity=0.2260
Epoch 36: Loss=172.8514, Sparsity=0.2405
Epoch 37: Loss=170.8513, Sparsity=0.2551
Epoch 38: Loss=168.7714, Sparsity=0.2698
Epoch 39: Loss=167.4562, Sparsity=0.2850
Epoch 40: Loss=165.6983, Sparsity=0.3008
Epoch 41: Loss=164.5605, Sparsity=0.3176
Epoch 42: Loss=161.3933, Sparsity=0.3356
Epoch 43: Loss=159.4453, Sparsity=0.3554
Epoch 44: Loss=158.7381, Sparsity=0.3776
Epoch 45: Loss=156.1692, Sparsity=0.4025
Epoch 46: Loss=157.7995, Sparsity=0.4290
Epoch 47: Loss=152.7207, Sparsity=0.4540
Epoch 48: Loss=152.4090, Sparsity=0.4747
Epoch 49: Loss=151.0381, Sparsity=0.4903
Epoch 50: Loss=150.3722, Sparsity=0.5023

Final Results:
Lambda  Accuracy        Sparsity
0.001   78.04%          0.37
0.005   78.97%          0.44
0.01    79.07%          0.50
```